STARHACKADEMINT 2023 WRITE-UPS : REVERSE

Redhpm
redhpm@hackademint.org

Version 1.
Merci de me rapporter toute erreur (typo/bêtise) sur discord @redhpm.
Si des explications ne sont pas claires, vous pouvez me poser des questions aussi.
Je ferai une v2 si certaines remarques reviennent.

Table des matières

1 Challenges de difficulté : Intro **2**

1.1 Cave à vin : Gewürzt (1/4) 3

2 Challenges de difficulté : Facile **10**

2.1 Cave à vin : Riesling (2/4) 11

2.2 Keygenme 1 16

2.3 Keygenme 2 18

3 Challenges de difficulté : Moyen **23**

3.1 Mais t'es point NET Baptiste ! 24

3.2 Cave à vin : Sylvaner (3/4) 31

3.3 Le cadeau Mystère 36

4 Challenges de difficulté : Difficile **41**

4.1 Cave à vin : Muscat (4/4) 42

4.2 Disques rayés 46

1 Challenges de difficulté : Intro

Il n'y a qu'un seul challenge de cette difficulté, son objectif est d'être le plus simple possible, afin de montrer le principe (dans les grandes lignes), de la catégorie. Dans notre contexte nous ferons du "software reverse engineering", c'est à dire comprendre en profondeur un logiciel, qui le plus souvent est le produit d'une compilation. En effet un CPU d'ordinateur ne comprend qu'un nombre limité d'instructions. Notre objectif est de faire un travail inverse à un compilateur, remonter le temps à la recherche d'une représentation plus "haut niveau", avec plus d'abstractions, de la logique du programme.

1.1 Cave à vin : Gewürzt (1/4)

Commençons par lire l'énoncé.

FIGURE 1 – Énoncé du challenge.

Dans 90 % des cas, un challenge de reverse en CTF se présente sous la forme de ce qu'on appelle un "crackme". Il s'agit simplement d'un programme qui demande un mot de passe, et qui vérifie ce mot de passe. Le but est de trouver ce mot de passe. Ce challenge ne fait donc pas exception.

Il existe peut être 50 solutions différentes à ce challenge, qui impliquent divers niveaux de compréhension. Je vais tenter ici une approche très élémentaire, de sorte à ce que vous puissiez comprendre toute la logique. Je ne vais pas utiliser de logiciels avancés, nous verrons ça pour d'autres challenges.

Le premier réflexe important est de savoir à quel fichier on a à faire. Une fois téléchargé, nous utilisons la commande linux *file*, qui nous apprend un grand nombre d'informations précieuses. La collecte d'informations est la première étape de tout travail de reverse. C'est une étape cruciale, car savoir obtenir les informations importantes efficacement, c'est un gain de temps énorme. Et comme toujours, le plus grand ennemi du reverse engineer, c'est le temps.

Quel est donc ce fichier ?

```
┌─redhpm@debian in ~/Downloads
└─$ file caveavin1
caveavin1: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, not stripped
```

Comme prévu il s'agit d'un fichier exécutable, au format ELF. Ce format est le standard pour les binaires sous linux. On apprend également que le binaire n'est pas strippé, c'est à dire que les symboles originaux (noms de variables/fonctions etc..) sont encore présents dans le binaire.

Lançons le binaire pour voir de quoi il retourne.

```
┌─redhpm@debian in ~/Downloads
└─$ chmod +x caveavin1
┌─redhpm@debian in ~/Downloads
└─$ ./caveavin1
Usage : ./caveavin1 [PASSWORD]%
┌─redhpm@debian in ~/Downloads
└─$ ./caveavin1 toto
Too Bad...%
```

Le programme prend donc en entrée le premier paramètre, puis d'une façon ou d'une autre, effectue un test sur cette entrée. Pour savoir quel est ce test, il faut regarder le code exécuté par le binaire.

Pour cela je vais faire quelque chose que je vais ne plus jamais faire dans ce PDF, utiliser *objdump*. Il s'agit d'un outil de la suite binutils, que permet notamment de désassembler le code exécutable d'un binaire.

```
┌─redhpm@debian in ~/Downloads
└─$ objdump -M intel -d caveavin1
```

Le flag $-d$ permet de désassembler, le flag $-M$ permet de choisir une syntaxe pour le langage assembleur. Ici je choisi la syntaxe Intel, qui est supérieure en tout point à la syntaxe AT&T par défaut.

Scrollons un peu jusqu'à la fonction *main*, dont le symbole est encore présent, puisque le binaire n'est pas strippé.

```
0000000000401146 <main>:
  401146:       55                      push   rbp
  401147:       48 89 e5                mov    rbp,rsp
  40114a:       48 83 ec 10             sub    rsp,0x10
  40114e:       89 7d fc                mov    DWORD PTR [rbp-0x4],edi
  401151:       48 89 75 f0             mov    QWORD PTR [rbp-0x10],rsi
  401155:       83 7d fc 01             cmp    DWORD PTR [rbp-0x4],0x1
  401159:       7f 19                   jg     401174 <main+0x2e>
  40115b:       bf 08 20 40 00          mov    edi,0x402008
  401160:       b8 00 00 00 00          mov    eax,0x0
  401165:       e8 c6 fe ff ff          call   401030 <printf@plt>
  40116a:       bf 01 00 00 00          mov    edi,0x1
  40116f:       e8 dc fe ff ff          call   401050 <exit@plt>
  401174:       48 8b 45 f0             mov    rax,QWORD PTR [rbp-0x10]
  401178:       48 83 c0 08             add    rax,0x8
  40117c:       48 8b 00                mov    rax,QWORD PTR [rax]
  40117f:       48 89 c6                mov    rsi,rax
  401182:       bf 27 20 40 00          mov    edi,0x402027
  401187:       e8 b4 fe ff ff          call   401040 <strcmp@plt>
  40118c:       85 c0                   test   eax,eax
  40118e:       75 19                   jne    4011a9 <main+0x63>
  401190:       bf 44 20 40 00          mov    edi,0x402044
  401195:       b8 00 00 00 00          mov    eax,0x0
  40119a:       e8 91 fe ff ff          call   401030 <printf@plt>
  40119f:       bf 00 00 00 00          mov    edi,0x0
  4011a4:       e8 a7 fe ff ff          call   401050 <exit@plt>
  4011a9:       bf 48 20 40 00          mov    edi,0x402048
  4011ae:       b8 00 00 00 00          mov    eax,0x0
  4011b3:       e8 78 fe ff ff          call   401030 <printf@plt>
  4011b8:       bf 00 00 00 00          mov    edi,0x0
  4011bd:       e8 8e fe ff ff          call   401050 <exit@plt>
```

Chaque ligne est une instruction, composée d'un "opérateur", sous la forme d'une mnémonique, et d'opérandes. La mnémonique permet de savoir quelle est l'opération en cours, par exemple une addition, un déplacement, un appel de fonction etc.. Les opérandes peuvent être de 3 types (le plus souvent, il y en a un 4ème qui ne nous intéresse pas ici). Ces "types" sont en réalité ce qu'on appelle le mode d'adressage.
Il y a tout d'abord les valeurs immédiates, qui sont des nombres.
Il y a les registres, qui sont des petites "variables", directement sauvegardées dans des emplacements du CPU. Ainsi leur accès est très rapide, mais également limité, par leur nombre et leur capacité de stockage.
Enfin il y a les adresses mémoire, qui sont simplement des adresses vers des emplacement de la mémoire virtuelle. Cette mémoire virtuelle pouvant correspondre à divers types de mémoire (le plus souvent la RAM).

Puisqu'il s'agit du challenge d'introduction, je vais maintenant détailler chacune des lignes de ce programme. Ici la fonction est extrêmement courte et simple donc c'est possible. Mais très vite vous allez devoir faire le tri dans ce que vous voulez analyser en détail et le reste.

```
push rbp
mov rbp, rsp
sub rsp, 0x10
```

Ce morceau de code est très classique, c'est ce qu'on appelle le prologue de la fonction. Il permet de mettre en place la stack frame de la fonction. La stack frame, c'est un espace de la stack qui est alloué pour les variables locales à la fonction. La stack c'est un espace mémoire sous forme d'une pile (la structure de donnée), qui permet entre autre d'avoir un espace de travail pour chacune des fonctions. Ici on manipule les registres *rbp* et *rsp*, qui délimitent la stack frame. Ils contiennent donc des adresses mémoire.
Ces lignes ont donc pour effet de :

— Pousser la valeur de rbp sur la stack, de sorte à la sauvegarder pour plus tard.
— Remplacer rbp par rsp. Ainsi le haut de la stack frame de la fonction appelante deviens le bas de la stack frame de la fonction appelée.
— Augmente la taille de la stack frame de 16, c'est à dire 2 qwords, 2 valeurs de 64 bits. Il s'agit là des espaces prévus pour 2 variables locales de cette taille. Notez qu'il s'agit d'un *sub*, c'est à dire une soustraction. En effet, le haut de la stack est pour les adresses les plus basses. On dit que la stack grandit vers le bas.

```
mov DWORD PTR [rbp-0x4], edi
mov QWORD PTR [rbp-0x10], rsi
```

Ici on sauvegarde *edi* et *rsi* à 2 emplacements sur la stack, dans la range qui avait été libérée pendant le prologue. Il s'agit donc des variables locales. En 64 bits, *rdi* et *rsi* sont les arguments de *main*, c'est à dire *argc* et *argv*. Si vous n'avais jamais fait de C, *argc* est un entier qui correspond au nombre d'arguments, *argv* est une adresse vers une suite d'adresse vers le premier caractère de chaque argument. Si ce concept ne vous semble pas clair, une explication plus détaillée se trouve un peu plus tard dans l'explication du programme.

```
cmp DWORD PTR [rbp-0x4], 0x1
jg 401174 <main+0x2e>
```

Ici on compare la valeur de argc (voir la section d'avant) avec 1. Si argc > 1, on saute plus loin dans le programme.

Le saut skip donc une partie du code qui est :

```
mov edi, 0x402008
mov eax, 0X0
call 401030 <printf@plt>
mov edi, 0x1
call 401050 <exit@plt>
```

Ici il s'agit de l'appel à 2 fonctions, d'abord *printf* puis *exit*. Les appels sont signifiés par les instructions *call*. Les autres instructions ont comme rôle de mettre en place les arguments pour les fonctions appelées. Avant printf, on met dans *edi*, l'adresse d'une chaîne de caractères (qui n'est en C qu'une succession de bytes, terminé par un byte nul.). Dans *rax*, ont met 0 car il s'agit du nombre d'arguments supplémentaires pour printf. Ici il n'y en pas. Ensuite exit prend un argument, qui est le code de sortie. Un nombre différent de 0 est par convention une erreur.

Pour récapituler, si le nombre d'arguments est <= 1, alors on affiche un message et on quitte le programme. Quel est ce message ? Il suffit d'aller voir ce qu'il y a à l'adresse 0x402008.

Par exemple affichons la section *.rodata*, qui sont les données en lecture seule dans un fichier ELF.

```
┌─redhpm@debian in ~/Downloads
└─$ objdump -s -j .rodata caveavin1

caveavin1:     file format elf64-x86-64

Contents of section .rodata:
 402000 01000200 00000000 55736167 65203a20  ........Usage : 
 402010 2e2f6361 76656176 696e3120 5b504153  ./caveavin1 [PAS
 402020 53574f52 445d0053 7461727b 43306d70  SWORD].Star{C0mp
 402030 34726131 73306e5f 44335f73 7452316e  4ra1s0n_D3_stR1n
 402040 67537d00 47472100 546f6f20 4261642e  gS}.GG!.Too Bad.
 402050 2e2e00                               ...
```

On remarque que le message affiché est "Usage : ./caveavin1 [PASSWORD]". C'est parfaitement logique et en adéquation avec le test que nous avions effectué au début. Une précision s'impose tout de même, quand dans un programme on écrit par exemple :

```
./programme arg1 arg2 arg3
```

Il y a 4 arguments en tout :
— Le premier est *./programme*
— Le deuxième est $arg1$
— Le troisième est $arg2$
— Le dernier est $arg3$
Ainsi, le programme $caveavin1$ requiert bien au minimum 2 arguments !

Continuons l'explication du programme.

```
mov rax, QWORD PTR [rbp-0x10]
add rax, 0x8
mov rax, QWORD PTR [rax]
```

Ici, on récupère $argv$ et on fait quelques opérations dessus. Rappelez vous, $argv$, est de type $char**$. Il s'agit donc d'une adresse vers une adresse vers un caractère. Cette adresse est celle de l'adresse du premier argument. Toutes les adresses des arguments sont consécutives en mémoire, et chaque adresse fait 8 bytes de large. Ainsi, en ajoutant 8 à l'adresse de l'adresse du premier argument, on obtient l'adresse de l'adresse du 2 ème argument. Relisez cette phrase autant de fois que nécessaire à sa compréhension. Il est crucial de comprendre ce concept.

La dernière instruction de cet extrait ne fait que suivre l'adresse en rax. Autrement dit, on passe de l'adresse de l'adresse du deuxième argument dans rax à l'adresse du deuxième argument. Ainsi, à ce niveau dans le code, rax contient l'adresse du mot de passe que vous avez rentré.

```
mov rsi, rax
mov edi, 0x402027
call 401040 <strcmp@plt>
```

Ici on appelle *strcmp*, qui est une fonction qui compare 2 chaînes de caractères. Le premier argument est rdi, qui est une adresse fixe, le deuxième est esi qui est notre adresse calculée précédemment, donc notre mot de passe. Le retour de la fonction se fait dans rax. Cela est très intéressant, une comparaison entre le mot de passe et une chaîne est effectuée.

Allons voir quelle est cette chaîne de la même façon que nous étions allé voir la première.

```
┌─redhpm@debian in ~/Downloads
└─$ objdump -s -j .rodata caveavin1

caveavin1:     file format elf64-x86-64

Contents of section .rodata:
 402000 01000200 00000000 55736167 65203a20  ........Usage :
 402010 2e2f6361 76656176 696e3120 5b504153  ./caveavin1 [PAS
 402020 53574f52 445d0053 7461727b 43306d70  SWORD].Star{C0mp
 402030 34726131 73306e5f 44335f73 7452316e  4ra1s0n_D3_stR1n
 402040 67537d00 47472100 546f6f20 4261642e  gS}.GG!.Too Bad.
 402050 2e2e00                               ...
```

Cette fois ci, avec l'offset 0x402027, nous obtenons la chaîne : "Star{C0mp4ra1s0n_D3_stR1ngS}". Vous avez sûrement deviné la suite, mais continuons notre analyse tout de même. Je vais aller un peu plus vite puisque nous avons déjà expliqué tous les concepts.

```
test eax, eax
jne 4011a9 <main+0x63>
mov edi, 0x402044
mov eax, 0x0
call 401030 <printf@plt>
mov edi, 0x0
call 401050 <exit@plt>
mov edi, 0x402048
mov eax, 0x0
call 401030 <printf@plt>
mov edi, 0x0
call 401050 <exit@plt>
```

La première instruction teste si *eax* vaut 0, c'est à dire si les 2 chaînes sont égales. L'instruction suivante va donc jump si ce n'est pas le cas. Sinon on continue. Les 2 blocs potentiellement exécutés sont quasiment identiques. Ils affichent un message puis quittent. On peut aller voir les messages en lisant la section *.rodata*, comme nous l'avons fait avant. Il s'agit ou bien d'un message de défaite : "Too Bad...", ou de réussite : "GG !".
Voici comment fonctionne le programme de ce challenge. On peut donc maintenant vérifier notre analyse en testant le mot de passe :

```
┌─redhpm@debian in ~/Downloads
└─$ ./caveavin1 Star{C0mp4ra1s0n_D3_stR1ngS}
GG!%
```

2 Challenges de difficulté : Facile

Passons maintenant aux challenges faciles. Je vais ici introduire des logiciels plus avancés, qui permettent d'accélérer votre méthodologie de reverse. Il est très important de maîtriser des outils puissants quand la complexité des programmes à analyser augmente.

2.1 Cave à vin : Riesling (2/4)

Commençons par lire l'énoncé.

Cave à vin : Riesling (2/4)

100

easy

Ce challenge a été tiré du StarHackademINT 2022, puis modifié

J'ai appelé un ami qui débute en cyber, il m'a dit qu'il a réparé la serrure de ma magnifique cave à vin. Il faudrait essayer pour voir si il a bien fait son travail...

Challenge original : Crackme2 de Redhpm

Auteur du vol : Sckathach

caveavin2

FIGURE 2 – Énoncé du challenge.

On est en présence d'un binaire ELF 64 bits, qui cette fois a été strippé. Cela signifie que les symboles dont on peut se passer ont été retirés. Je précise bien cela puisque un certain nombres de symboles restent.

Pour ce challenge, je vais utiliser un logiciel du nom de *radare*2. C'est un framework de reverse engineering très puissant, mais qui peut sembler austère de prime abord. Le binaire à analyser est assez simple, donc une utilisation très basique du logiciel suffit.

Tout d'abord j'ouvre le binaire dans r2.

```
┌─redhpm@debian in ~/Downloads
└─$ r2 caveavin2
WARN: Relocs has not been applied. Please use `-e bin.relocs.apply=true` or `-e bin.cache=true` next time
 -- Most of commands accept '?' as a suffix. Use it to understand how they work :)
[0x00401060]>
```

On se retrouve avec une interface en ligne de commande. Les commandes dans

r2 sont en général ce qui fait peur aux débutants. Il faut comprendre leur principe. Les commandes sont organisée sous forme d'un arbre, et chaque profondeur rajoute une lettre à la commande. Par exemple les commandes liées à l'analyse commencent par "a". Celles liées à l'affichage par "p" (pour print).

Commençons par lancer une analyse en profondeur du binaire avec la commande *aaa*.

```
[0x00401060]> aaa
INFO: Analyze all flags starting with sym. and entry0 (aa)
INFO: Analyze imports (af@@@i)
INFO: Analyze entrypoint (af@ entry0)
INFO: Analyze symbols (af@@@s)
INFO: Recovering variables
INFO: Analyze all functions arguments/locals (afva@@@F)
INFO: Analyze function calls (aac)
INFO: Analyze len bytes of instructions for references (aar)
INFO: Finding and parsing C++ vtables (avrr)
INFO: Analyzing methods
INFO: Recovering local variables (afva)
INFO: Type matching analysis for all functions (aaft)
INFO: Propagate noreturn information (aanr)
INFO: Use -AA or aaaa to perform additional experimental analysis
```

On peut ensuite par exemple, lister les fonctions, avec *afl* (pour analyze functions list, logique !).

```
[0x00401060]> afl
0x00401030    1      6 sym.imp.strlen
0x00401040    1      6 sym.imp.printf
0x00401050    1      6 sym.imp.exit
0x00401060    1     33 entry0
0x00401146   53   1245 main
0x00401140    5     55 entry.init0
0x00401110    3     32 entry.fini0
0x004010a0    4     31 fcn.004010a0
```

La convention de nommage des fonctions dans r2 nous donne déjà beaucoup d'informations. Celles qui commencent par *sym.imp* sont des fonctions importées, donc qui viennent de librairies dynamiquement liées. $fcn.004010a0$ est une fonction

sans nom. Cela vient du fait que le binaire est strippé. Le symbole *main* n'est d'ailleurs pas présent, mais r2 est capable de retrouver la fonction main pendant son analyse. Il nous l'a donc gentiment renommée.

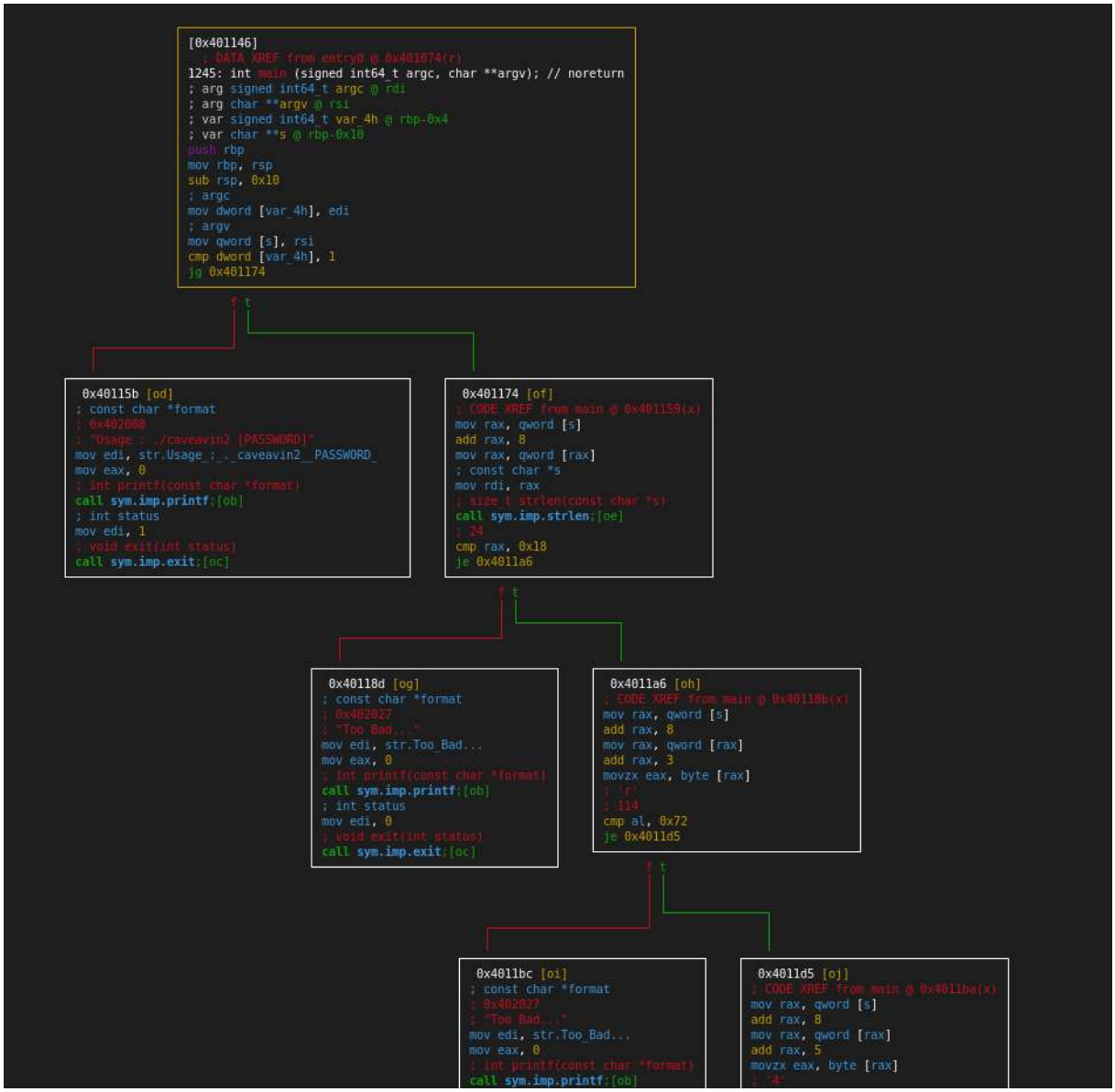
Dans radare2, on appelle *seek*, l'adresse courante sur laquelle on est placé. Il s'agit de celle qui est indiquée dans le prompt. On peut modifier ce seek avec la commande "s".

Une fois placé sur main, on peut afficher l'assembleur de la fonction avec *pdf*.

```
[0x00401060]> s main
[0x00401146]> pdf
Do you want to print 348 lines? (y/N) y
            ; DATA XREF from entry0 @ 0x401074(r)
┌ 1245: int main (signed int64_t argc, char **argv); // noreturn
│           ; arg signed int64_t argc @ rdi
│           ; arg char **argv @ rsi
│           ; var signed int64_t var_4h @ rbp-0x4
│           ; var char **s @ rbp-0x10
│           0x00401146      55              push rbp
│           0x00401147      4889e5          mov rbp, rsp
│           0x0040114a      4883ec10        sub rsp, 0x10
│           0x0040114e      897dfc          mov dword [var_4h], edi     ; argc
│           0x00401151      488975f0        mov qword [s], rsi          ; argv
│           0x00401155      837dfc01        cmp dword [var_4h], 1
│       ┌─< 0x00401159      7f19            jg 0x401174
│       │   0x0040115b      bf08204000      mov edi, str.Usage_:_._caveavin2__PASSWORD_ ; 0x402008
│       │   0x00401160      b800000000      mov eax, 0
│       │   0x00401165      e8d6feffff      call sym.imp.printf         ; int printf(const char *f
│       │   0x0040116a      bf01000000      mov edi, 1                  ; int status
│       │   0x0040116f      e8dcfeffff      call sym.imp.exit           ; void exit(int status)
│       │   ; CODE XREF from main @ 0x401159(x)
│       └─> 0x00401174      488b45f0        mov rax, qword [s]
│           0x00401178      4883c008        add rax, 8
│           0x0040117c      488b00          mov rax, qword [rax]
│           0x0040117f      4889c7          mov rdi, rax                ; const char *s
```

On peut souvent bien mieux visualiser avec le mode "panel" de r2. Celui ci va nous afficher le CFG (control flow graph) de la fonction. Le control flow graph est un graph dont les nodes sont les blocs de bases (suite d'instructions sans branchement), et les edges sont les transitions (jumps).

On passe en mode panel avec la commande ”VV”.



Cela permet de visualiser beaucoup plus simplement la logique du programme. Le programme vérifie la longueur du mot de passe avec la fonction strlen. Ensuite les caractères du mot de passe sont vérifiés 1 par 1, dans un ordre arbitraire.

Tous les blocs ont la même structure. Remarquez que radare2 a effectué quelques modifications afin de nous rendre la lecture plus simple. Des noms de variables ont été crées, il y a de la coloration syntaxique, des commentaires sont ajoutés etc...

Voici la structure d'un bloc, qui check 1 caractère du mot de passe.

```
mov rax, qword [s]
add rax, 8
mov rax, qword [rax]
add rax, XX
movzx rax, byte [rax]
cmp al, XX
je XXXXXXXX
```

Il s'agit d'un code très similaire à ce qu'on avait vu lors de l'intro, La seule différence viens d'un add supplémentaire qui permet de déplacer le pointeur vers un autre caractère que le premier du mot de passe. Avec seulement 24 checks, il est possible de le faire à la main. Afin de faciliter le solve, la commande suivante vous permet de tout avoir sous les yeux.

```
pif | grep -B2 al, | grep -v mov
```

```
[0x00401146]> pif | grep -B2 al, | grep -v mov
add rax, 3
cmp al, 0x72
--
add rax, 5
cmp al, 0x34
--
add rax, 0x15
cmp al, 0x67
--
add rax, 0x14
cmp al, 0x6e
--
add rax, 0x17
cmp al, 0x7d
--
add rax, 4
cmp al, 0x7b
--
```

Finalement le flag est : "Star{4_m01_l3_r13SL1ng!}"

2.2 Keygenme 1

Commençons par lire l'énoncé.

Keygenme 1

496

easy

Vous devez developper un programme qui genère des clés valides pour un serial quelconque. Ensuite vous pourrez résoudre l'épreuve en ligne.

PS : Le binaire en utilisation interactive fait foi.

Auteur : Redhpm

nc challenges.hackademint.org 32039

keygenme1

FIGURE 3 – Énoncé du challenge.

Je vais maintenant utiliser des logiciels plus avancés et considérer que vous maîtriser un minimum l'assembleur et le langage C.

Ouvrons le binaire donné avec IDA (il s'agit du logiciel que je vais le plus utiliser dans ce PDF).

```
puts("Input serial:");
fgets(s, 32, stdin);
if ( !strchr(s, 10) )
{
  do
    v12 = getchar();
  while ( v12 != 10 && v12 != -1 );
}
puts("Input key:");
fgets(s2, 32, stdin);
s[0] ^= *(_BYTE *)off_404050;
sub_401166(s);
HIBYTE(v7) = 0;
for ( i = 1; i <= 31; ++i )
{
  s[i] ^= s[i - 1];
  sub_401166(&s[i]);
}
HIBYTE(v11) = 0;
if ( !strcmp(s, s2) )
  puts(":)");
else
  puts(":(");
return 0LL;
```

Les 2 fgets permettent de récupérer le serial et le clé. Le retour à la ligne n'est pas enlevé du serial. Il fait une taille de 32 bytes max.

Le serial subit une modification puis est comparée à la clé proposée.

Tout d'abord le premier caractère est xoré avec un byte qui se situe dans la section rodata. En double cliquant sur off_404050 on obtient la valeur pointée. Il s'agit d'un $0x90$. Puis chaque caractère est modifié par la fonction sub_401166. Celle-ci permet de transformer un byte en lettre majuscule avec le calcul suivant :

```
c = c % 26 + 65
```

Ensuite les caractères sont xorés avec le byte précédant.

Il suffit de ré-implémenter cet algorithme en python, sans oublier les éventuels $\backslash n$ et de pad à 32 byte le serial.
J'effectue la communication TCP avec la bibliothèque python *nclib*.

```python
import nclib

nc = nclib.Netcat('challenges.hackademint.org', 32039)

def key(serial):
    rkey = chr((0x90 ^ ord(serial[0]))%26 + 0x41)
    for i in range(1, 32):
        rkey += chr((ord(rkey[i - 1]) ^ ord(serial[i]))% 26 + 0x41)
    return rkey



while 1:
    content = nc.recv_until(b'>>> ')
    print(content.decode())
    try:
        ser = content.split(b'suivant : ')[1].split(b' ?')[0].decode()
    except:
        exit()

    if len(ser) != 32:
        ser += '\n' + (31 - len(ser))*'\x00'

    k = key(ser)[:31].encode()
    nc.send_line(k)
```

2.3 Keygenme 2

Commençons par lire l'énoncé.

Keygenme 2

500

easy

Vous devez developper un programme qui genère des clés valides pour un serial quelconque. Ensuite vous pourrez résoudre l'épreuve en ligne.

PS : Le binaire en utilisation interactive fait foi.

Auteur : Redhpm

nc challenges.hackademint.org 30821

keygenme2

FIGURE 4 – Énoncé du challenge.

Ouvrons le binaire dans IDA.

```
puts("Input serial:");
fgets(s, 32, stdin);
if ( !strchr(s, 10) )
{
  do
    v21 = getchar();
  while ( v21 != 10 && v21 != -1 );
}
puts("Input key: (hexform)");
fgets(v8, 65, stdin);
if ( !strchr(v8, 10) )
{
  do
    v21 = getchar();
  while ( v21 != 10 && v21 != -1 );
}
v16 = 0;
__isoc99_sscanf(
  v8,
  "%08x%08x%08x%08x%08x%08x%08x%08x",
  &v4,
  (char *)&v4 + 4,
  &v5,
  (char *)&v5 + 4,
  &v6,
  (char *)&v6 + 4,
  &v7,
  (char *)&v7 + 4);
if ( (unsigned int)sub_401166(s, &v4, &loc_401276) )
  puts(":(");
else
  puts(":)");
return 0LL;
```

Le serial et la clé sont récupérés de la même façon que pour le challenge 1. Cette fois-ci on nous demande la clé sous forme hexadécimale. On voit la justification de cette demande immédiatement dans le code source, puisque un *sscanf* récupère directement les codes hexadécimaux (8 fois 4 = 32 pour être précis) pour les stocker dans des variables. Ici on peut se douter qu'il y avait un tableau initialement. Sous IDA, j'utilise la touche Y pour convertir la variable en *char*[32]. Le code est un peu plus lisible ce faisant.

```
v13 = 0;
__isoc99_sscanf(
  v5,
  "%08x%08x%08x%08x%08x%08x%08x%08x",
  v4,
  &v4[4],
  &v4[8],
  &v4[12],
  &v4[16],
  &v4[20],
  &v4[24],
  &v4[28]);
if ( (unsigned int)sub_401166(s, v4, (char *)&loc_401276) )
  puts(":(");
else
  puts(":)");
return 0LL;
```

Attention, le format de scan fait en sorte d'inverser l'ordre des octets par groupes de 4 (c'est ce qu'on appelle *l'endianness* ou boutisme en français).
Ici j'ai rentré AABBCCDD 8 fois en tant que clé, et pourtant dans le tableau v4, on a DDCCBBAA etc...
NB : il s'agit du mode debug de IDA.

```
}
v5[64] = 0;
__isoc99_sscanf(
  v5,
  "%08x%08x%08x%08x%08x%08x%08x%08x",
  v4,
  &v4[4]
  &v4char v4[32]; // [rsp+0h] [rbp-A0h] BYREF
  &v4('\xDD','\xCC','\xBB','\xAA','\xDD','\xCC','\xBB','\xAA','\xDD','\xCC','\xBB','\xAA','\xDD','\xCC','\xBB
  &v4[16],
  &v4[20],
  &v4[24],
```

Tout dépend donc de la fonction *sub_401166*, que je renomme *chk* avec la touche N. J'en profite pour retyper les arguments de la fonction.
IDA donne le résultat suivant.

```
__int64 __fastcall chk(unsigned __int8 *a1, unsigned __int8 *a2, unsigned __int8 *a3)
{
  __int64 v3; // kr00_8
  int i; // [rsp+20h] [rbp-8h]
  unsigned int v6; // [rsp+24h] [rbp-4h]

  v6 = 0;
  for ( i = 0; i <= 15; ++i )
  {
    v3 = a1[2 * i] + a1[2 * i + 1] + a2[2 * i] + a2[2 * i + 1];
    if ( (unsigned __int8)(HIBYTE(v3) + a1[2 * i] + a1[2 * i + 1] + a2[2 * i] + a2[2 * i + 1]) - HIBYTE(HIDWORD(v3)) != a3[i]
      || (unsigned __int8)(a1[2 * i] * a1[2 * i + 1]) != a2[2 * i] )
    {
      v6 = 1;
    }
  }
  return v6;
}
```

On observe ici un ensemble de conditions à passer, qui sont facilement satisfaites. Notons tout de même une bizarrerie de décompilation au niveau de la première condition. Les macros appliquées à v3 s'annulent, en effet l'octet de poids fort de v3, est égal à l'octet de poids fort du dword de poids fort de v3...

On peut donc résumer les conditions comme :

```
Pour chaque paire  A, B du serial, et C, D de la clé :
    A*B = C ET
    A + B + C + D = X avec X fixé.
```

On voit bien qu'avec A B et X connus, on retrouve C et D facilement. Il ne nous reste plus qu'à connaître X pour chaque paire. Il est donné en tant que tableau par le 3ème argument de *chk*.

Dans *main*, *chk* est lancé avec &*loc_401276* comme argument. Il suffit de récupérer les données à cet endroit (16 octets, 1 par paire du serial/clé.) Si on double clique sur l'argument sous IDA, il semble ne rien se passer.
En réalité IDA ne bouge pas car vous êtes déjà à l'endroit où il y a les données, puisqu'il s'agit d'un offset dans la section exécutable du binaire. Le symbole *main* est en 0x401275, nous avons donc ici l'offset $main+1$. Pour récupérer les données il y a plusieurs façon, j'ai décidé de passer par radare2, puisque une des commandes est très pratique pour ensuite faire un script python.

Je lance donc r2, et je fais :

```
aaa
pcp 16 @ main+1
```

Ce qui donne :

```
[0x00401080]> aaa
INFO: Analyze all flags starting with sym. and entry0 (aa)
INFO: Analyze imports (af@@@i)
INFO: Analyze entrypoint (af@ entry0)
INFO: Analyze symbols (af@@@s)
INFO: Recovering variables
INFO: Analyze all functions arguments/locals (afva@@@F)
INFO: Analyze function calls (aac)
INFO: Analyze len bytes of instructions for references (aar)
INFO: Finding and parsing C++ vtables (avrr)
INFO: Analyzing methods
INFO: Recovering local variables (afva)
INFO: Type matching analysis for all functions (aaft)
INFO: Propagate noreturn information (aanr)
INFO: Use -AA or aaaa to perform additional experimental analysis
[0x00401080]> pcp 16 @ main+1
import struct
buf = struct.pack ("16B", *[
0x48,0x89,0xe5,0x48,0x81,0xec,0xa0,0x00,0x00,0x00,0x48,
0xc7,0x45,0xd0,0x00,0x00])
[0x00401080]>
```

Pour réussir le challenge, j'utilise encore une fois nclib pour me connecter en TCP.

```python
import nclib
import struct
buf = struct.pack ("16B", *[
0x48,0x89,0xe5,0x48,0x81,0xec,0xa0,0x00,0x00,0x00,0x48,
0xc7,0x45,0xd0,0x00,0x00])


nc = nclib.Netcat('challenges.hackademint.org', 30821)

def key(serial):
    serial = [ord(c) for c in serial]
    s = ''
    key = [0] * 32
    for i in range(16):
        key[2 * i] = (serial[2 * i] * serial[2 * i + 1]) % 256
        key[2 * i + 1] = (buf[i] - serial[2 * i] - serial[2 * i + 1] - key[2 * i]) % 256

    for i in range(0, len(key), 4):
        s += f'{key[i+3]:02x}{key[i+2]:02x}{key[i+1]:02x}{key[i]:02x}'

    return s

```

```python

while 1:
    content = nc.recv_until(b'>>> ')
    print(content.decode())
    try:
        ser = content.split(b'suivant : ')[1].split(b' ?')[0].decode()
    except:
        exit()
    if len(ser) != 32:
        ser += '\n' + (31 - len(ser))*'\x00'

    k = key(ser).encode()
    nc.send_line(k)
```

3 Challenges de difficulté : Moyen

Pas grand chose à dire, on continue notre voyage dans la catégorie reverse.

3.1 Mais t'es point NET Baptiste !

Commençons par lire l'énoncé.

FIGURE 5 – Énoncé du challenge.

Le fichier donné est un exécutable windows, compilé depuis .net, comme nous l'apprend la commande *file*.

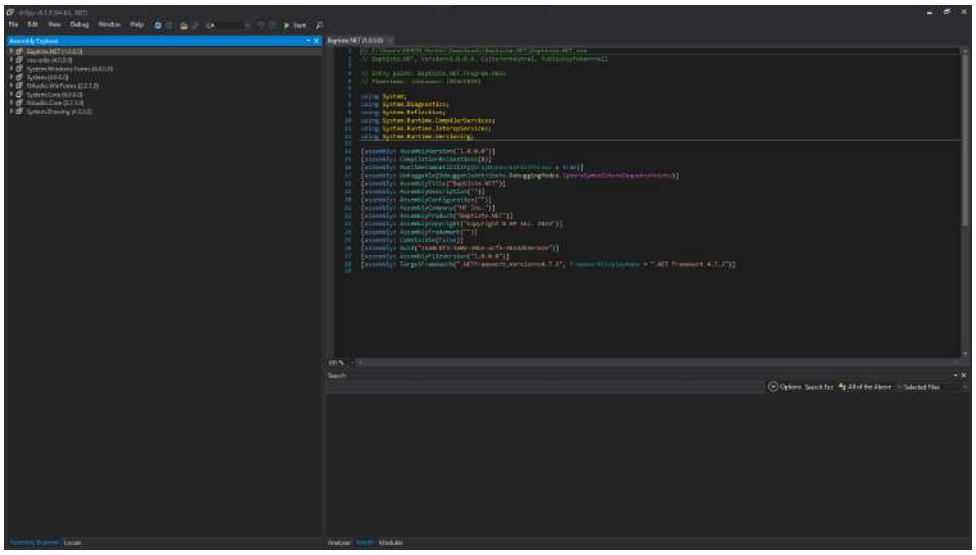
```
Baptiste.NET.exe: PE32 executable (GUI) Intel 80386 Mono/.Net assembly, for MS Windows
```

Allons donc sur windows pour lancer le binaire et voir de quoi il retourne.

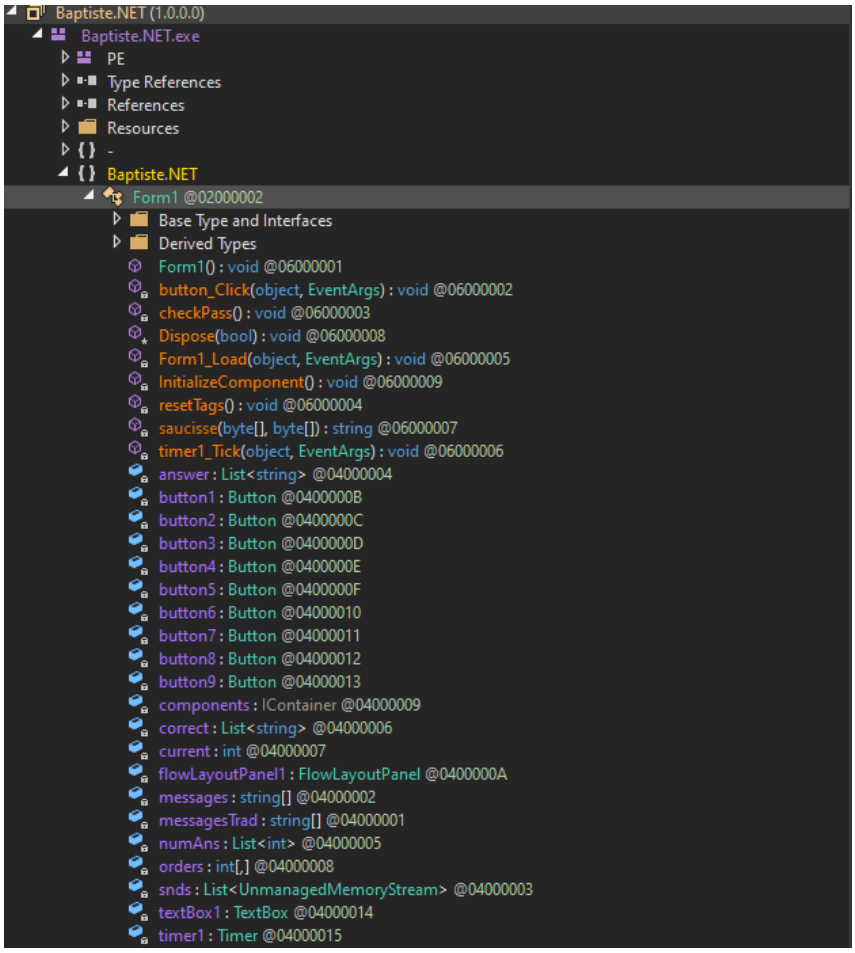
Nous sommes en présence de 9 boutons qui lancent des clips audio. Au bout de 6 clips lancés, un message apparaît en bas. Nous pouvons supposer qu'il suffit de trouver un code précis qui permet d'obtenir un autre message.

De façon générale, tout ce qui est compilé pour mono, peut être très facilement décompilé, avec un logiciel tel que *DNSpy*. En effet, le processus de reverse ne se fait pas sur du langage assembleur, mais sur un bytecode compris par la machine virtuelle de l'environnement .NET. Ce bytecode appelé *CIL* est facilement "reversable".

Quand on ouvre le fichier dans DNSpy on se retrouve sur cette interface, similaire à celle d'un IDE. En effet, le logiciel embarque un grand nombre de features, dont la décompilation n'est qu'une facette. Pour ce challenge, nous n'aurons pas besoin des fonctionnalités avancées.



Explorons donc l'arborescence située à gauche.



Plusieurs éléments semblent intéressants. Tout d'abord le binaire n'implémente qu'un seul objet, $Form1$. Une des méthodes de cette classe se nomme $checkPass$, commençons par l'analyser.

```
if (this.answer.SequenceEqual(this.correct))
{
    byte[] bytes = Encoding.ASCII.GetBytes(string.Join(" ", this.answer.ToArray()));
    byte[] key = SHA256.Create().ComputeHash(bytes);
    string str = Form1.saucisse(new byte[]
    {
        228,
        136,
        222,
        191,
        244,
        57,
        161,
        47,
        150,
        47,
        219,
        178,
        65,
        55,
        18,
        5,
        70,
        4,
        29,
        17,
        236,
        33,
        172,
        85,
        189,
        208,
        43,
        100,
        193,
        15,
        180,
        181,
        134,
        63,
        219,
        26,
        49,
        233,
        248,
        61,
        242,
        122,
        107,
        76,
        206,
        218,
        35,
        156
    }, key);
    this.textBox1.Text = "GG ! " + str;
    return;
}
this.textBox1.Text = "Ce mot de passe n'est .NET";
```

Il s'agit d"un simple if, qui permet de faire le choix entre 2 messages, un pour la réussite, l'autre pour la défaite. Nous apprenons ainsi qu'il suffit que notre réponse ($answer$) soit égale à la réponse correcte pour flag.

Notons que le message de réussite semble contenir le flag, celui ci est chiffré à partir d'une clé générée par le code PIN. La fonction *saucisse*, effectue le chiffrement. Il s'agit d'un simple chiffrement AES256-CBC. Autrement dit, pas d'arnaques, il faut bien trouver le code PIN.

Regardons maintenant ce qu'il se passe lorsque nous cliquons sur un bouton. La méthode *button_Click* semble adaptée.

```
private void button_Click(object sender, EventArgs e)
{
    int num = int.Parse(((Button)sender).Text);
    string item = this.messagesTrad[this.orders[this.current, num]];
    UnmanagedMemoryStream unmanagedMemoryStream = this.snds[this.orders[this.current, num]];
    unmanagedMemoryStream.Position = 0L;
    WaveOut waveOut = new WaveOut(WaveCallbackInfo.FunctionCallback());
    waveOut.Volume = 1f;
    waveOut.Init(new WaveFileReader(unmanagedMemoryStream));
    waveOut.Play();
    this.answer.Add(item);
    this.numAns.Add(num);
    this.current++;
    this.textBox1.Text = "Bruit n°" + this.current.ToString() + " joué";
    Button[] array = new Button[]
    {
        this.button1,
        this.button2,
        this.button3,
        this.button4,
        this.button5,
        this.button6,
        this.button7,
        this.button8,
        this.button9
    };
    for (int i = 0; i < 9; i++)
    {
        array[i].Enabled = false;
    }
    this.timer1.Start();
    if (this.current == this.correct.Count<string>())
    {
        this.checkPass();
        this.current = 0;
        this.answer.Clear();
        this.numAns.Clear();
    }
```

Parmi cette logique, nous décelons la construction du tableau *answer*, ce qui nous intéresse. Il s'agit d'une suite d'éléments obtenus à partir de 2 tableaux *orders*, et *messageTrad*.

Pour savoir quels sont ces tableaux, il suffit de cliquer dessus.
orders est un tableau de nombres, en 2 dimensions.
messagesTrad est un tableau de strings.

```
private int[,] orders = new int[,]
{
    {
        0,
        2,
        3,
        4,
        5,
        6,
        7,
        8,
        9
    },
    {
        1,
        3,
        5,
        7,
        9,
        2,
        4,
        6,
        8
    },
    {
        7,
        3,
        9,
        2,
        7,
        10,
        6,
        4,
        11
    },
    {
```

Nous savons comment est construit *answer* ; maintenant il faut savoir la valeur de *correct*.

Pour cela il suffit d'aller voir le constructeur de la classe :

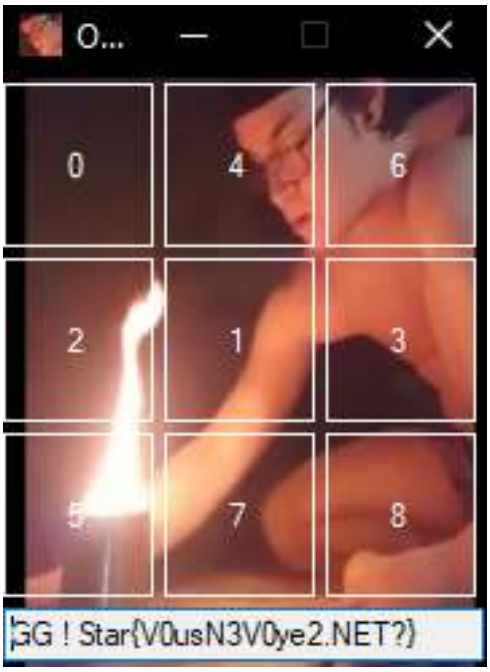
```
public Form1()
{
    string[] source = new string[]
    {
        this.messagesTrad[0],
        this.messagesTrad[7],
        this.messagesTrad[2],
        this.messagesTrad[1],
        this.messagesTrad[3],
        this.messagesTrad[5]
    };
    this.correct = source.ToList<string>();
    this.InitializeComponent();
}
```

Puisque pour construire *correct*, nous utilisons le même tableau *messagesTrad*, il suffit de savoir quel sont les boutons à appuyer pour obtenir les mêmes indices que ceux apparaissant dans le constructeur.

Pour retrouver le code PIN il suffit donc de trouver l'indice dans $orders[i]$ avec i allant de 0 à 5, de chacun des indices présents dans le constructeur.
On trouve :

033825.

En rentrant ce code dans le binaire on obtient le flag.



Bonus : Le déchiffrement du flag dans cyberchef !

https ://tinyurl.com/3akxktvr

3.2 Cave à vin : Sylvaner (3/4)

Commençons par lire l'énoncé.

Cave à vin : Sylvaner (3/4)

496

medium

Ce challenge a été tiré du StarHackademINT 2022, puis modifié

Décidement, mon ami a encore beaucoup de choses à apprendre. Il essaye néanmoins de se rattraper, cette fois grace à un système très imagé !

Challenge original : crackme3 de Redhpm

Tentative de vol par : Sckathach

Mais il était trop ~~nul~~ occupé donc c'est Redhpm qui l'a adapté pour cette année...

caveavin3

FIGURE 6 – Énoncé du challenge.

Nous sommes en présence d'un crackme, on commence à avoir l'habitude.

Ouvrons le binaire dans IDA.

```
1 void __fastcall __noreturn main(int a1, char **a2, char **a3)
2 {
3   unsigned __int8 v3; // bl
4   __int64 v4; // [rsp+18h] [rbp-28h]
5   FILE *stream; // [rsp+20h] [rbp-20h]
6   unsigned int i; // [rsp+2Ch] [rbp-14h]
7
8   if ( a1 <= 1 )
9   {
10    puts("Usage : ./caveavin3 [FILE]");
11    exit(1);
12  }
13  stream = fopen(a2[1], "rb");
14  if ( !stream )
15  {
16    printf("Can't open file %s\n", a2[1]);
17    exit(1);
18  }
19  fseek(stream, 0LL, 2);
20  v4 = ftell(stream);
21  fseek(stream, 0LL, 0);
22  if ( v4 != dword_F94A90 )
23  {
24    puts("Too bad...");
25    fclose(stream);
26    exit(0);
27  }
28  for ( i = 0; i < dword_F94A90; ++i )
29  {
30    v3 = getc(stream);
31    if ( ((unsigned __int8)(rand() % 255) ^ v3) != byte_404080[i] )
32    {
33      puts("Too Bad...");
34      fclose(stream);
35      exit(0);
36    }
37  }
38  puts("GG! The Flag is in the file you used to get here ;)");
39  fclose(stream);
40  exit(0);
41 }
```

La décompilation est quasi parfaite, en renommant les variables, et en appliquant le bon enum sur les constantes de *fseek*, nous obtenons un programme qui est probablement quasi identique au code source.

```
1  void __fastcall __noreturn main(int argc, char **argv, char **envp)
2  {
3    unsigned __int8 c; // bl
4    __int64 v4; // [rsp+18h] [rbp-28h]
5    FILE *fd; // [rsp+20h] [rbp-20h]
6    unsigned int i; // [rsp+2Ch] [rbp-14h]
7
8    if ( argc <= 1 )
9    {
10     puts("Usage : ./caveavin3 [FILE]");
11     exit(1);
12   }
13   fd = fopen(argv[1], "rb");
14   if ( !fd )
15   {
16     printf("Can't open file %s\n", argv[1]);
17     exit(1);
18   }
19   fseek(fd, 0LL, SEEK_END);
20   v4 = ftell(fd);
21   fseek(fd, 0LL, SEEK_SET);
22   if ( v4 != len )
23   {
24     puts("Too bad...");
25     fclose(fd);
26     exit(0);
27   }
28   for ( i = 0; i < len; ++i )
29   {
30     c = getc(fd);
31     if ( ((unsigned __int8)(rand() % 255) ^ c) != ciphered_data[i] )
32     {
33       puts("Too Bad...");
34       fclose(fd);
35       exit(0);
36     }
37   }
38   puts("GG! The Flag is in the file you used to get here ;)");
39   fclose(fd);
40   exit(0);
41 }
```

Ainsi le programme attend en argument le chemin vers un ficher, dont il va vérifier le contenu octet par octet. Le binaire embarque donc le fichier "mot de passe", sous une forme chiffrée. La méthode de chiffrement est un xor avec une clé aléatoire de la taille du message. C'est ce qu'on appelle un One Time Pad, un chiffrement incassable.
Mais si c'est incassable comment va-t-on pouvoir déchiffrer le message ?
Il y a tout simplement une erreur d'implémentation dans la génération de la clé. Ici la fonction *rand* de la libc a été utilisée, sans appel à *srand* au préalable. Par défaut, la seed est donc toujours la même, elle vaut 0 ! Le one time pad n'est incassable

qu'avec une source aléatoire parfaite, ce qui n'est clairement pas le cas ici.

Pour retrouver le fichier original il suffit donc :

— D'extraire le fichier chiffré.

— De reproduire le chiffrement, en codant en C par exemple, de sorte à ré-obtenir les mêmes valeurs "aléatoires".

J'extrais le fichier à l'aide de radare2, avec la commande :

```
pr 0xB90A0E @ 0x404080 > ciphered
```

Où 0xB90A0E est la taille, ciphered est le nom du fichier, 0x404080 est l'offset du début du fichier.

Voici un petit programme en C qui va effectuer le déchiffrement.

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char ** argv){

    if (argc < 2){
        printf("Usage : ./decipher [FILE]");
        exit(1);
    }

    FILE *fd;
    fd = fopen(argv[1], "rb");
    if (fd == NULL) {
        printf("Can't open file %s\n", argv[1] );
        exit(1);
    }
    fseek(fd, 0 , SEEK_END);
    long len = ftell(fd);
    fseek(fd, 0 , SEEK_SET);

    for (int i = 0; i < len; i++) {
        putc((unsigned char) getc(fd) ^ (unsigned char)(rand() % 255),
    stdout);
    }
    fclose(fd);
    exit(0);
}
```

Si on lance ce programme sur notre fichier chiffré, en re-dirigeant stdout, on se retrouve avec un fichier qui valide le crackme ! En lançant *file* sur le fichier on peut savoir comment ouvrir notre flag...

```
┌─redhpm@debian in ~/Downloads
└─$ ./decipher ciphered > deciphered
┌─redhpm@debian in ~/Downloads
└─$ ./caveavin3 deciphered
GG! The Flag is in the file you used to get here ;)
┌─redhpm@debian in ~/Downloads
└─$ file deciphered
deciphered: PNG image data, 4608 x 3072, 8-bit/color RGBA, non-interlaced
┌─redhpm@debian in ~/Downloads
```

Voici l'image avec le flag :

Le flag est donc : "Star{Ap3r4nD0M}"

3.3 Le cadeau Mystère

Commençons par lire l'énoncé.

FIGURE 7 – Énoncé du challenge.

Encore un crackme ! C'est parti.

Ouvrons le binaire dans IDA, et faisons un peu de ménage. J'ai renommé quelques variables et créé des enums de type "bitfields" pour les arguments de mmap.

```
 9  printf("Password:");
10  __isoc99_scanf("%20s", s);
11  if ( strlen(s) != 20 )
12  {
13    puts("No.");
14    exit(0);
15  }
16  key[0] = ptrace(PTRACE_TRACEME, 0LL, 0LL) + 161;
17  key[1] = ptrace(PTRACE_TRACEME, 0LL, 0LL) + 183;
18  key[2] = ptrace(PTRACE_TRACEME, 0LL, 0LL) + 224;
19  key[3] = ptrace(PTRACE_TRACEME, 0LL, 0LL) + 34;
20  shellcode_function = 0LL;
21  for ( i = 0; i < len; ++i )
22    shellcode[i] ^= LOBYTE(key[(int)i % 4]);
23  v3 = mmap(0LL, 0x12CuLL, PROT_EXEC|PROT_WRITE|PROT_READ, MAP_ANON|MAP_PRIVATE, -1, 0LL);
24  shellcode_function = (void (__fastcall *)(char *))v3;
25  *v3 = *(_QWORD *)shellcode;
26  *(_QWORD *)((char *)v3 + 292) = *(_QWORD *)&shellcode[292];
27  qmemcpy(
28    (void *)((unsigned __int64)(v3 + 1) & 0xFFFFFFFFFFFFFFF8LL),
29    (const void *)(shellcode - ((char *)v3 - ((unsigned __int64)(v3 + 1) & 0xFFFFFFFFFFFFFFF8LL))),
30    8LL * ((((_DWORD)v3 - (((_DWORD)v3 + 8) & 0xFFFFFFF8) + 300) & 0xFFFFFFF8) >> 3));
31  shellcode_function(s);
32  return 0LL;
```

Tout est assez clair sauf un gros détail, les arguments de qmemcpy. Regardons l'assembleur pour y voir plus clair.

Voici le bloc qui effectue la copie du shellcode.

```
mov     r9d, 0          ; offset
mov     r8d, 0FFFFFFFFh ; fd
mov     ecx, 22h ; '"'  ; flags
mov     edx, 7          ; prot
mov     esi, 12Ch       ; len
mov     edi, 0          ; addr
call    _mmap
mov     [rbp+shellcode_function], rax
mov     rax, [rbp+shellcode_function]
mov     rdx, rax
mov     eax, offset shellcode
mov     ecx, 12Ch
mov     rsi, [rax]
mov     [rdx], rsi
mov     esi, ecx
add     rsi, rdx
lea     rdi, [rsi+8]
mov     esi, ecx
add     rsi, rax
add     rsi, 8
mov     rsi, [rsi-10h]
mov     [rdi-10h], rsi
lea     rdi, [rdx+8]
and     rdi, 0FFFFFFFFFFFFFFF8h
sub     rdx, rdi
sub     rax, rdx
add     ecx, edx
and     ecx, 0FFFFFFF8h
shr     ecx, 3
mov     edx, ecx
mov     edx, edx
mov     rsi, rax
mov     rcx, rdx
rep movsq
lea     rax, [rbp+s]
mov     rdx, [rbp+shellcode_function]
mov     rdi, rax
call    rdx
mov     eax, 0
leave
retn
; } // starts at 401186
main endp
```

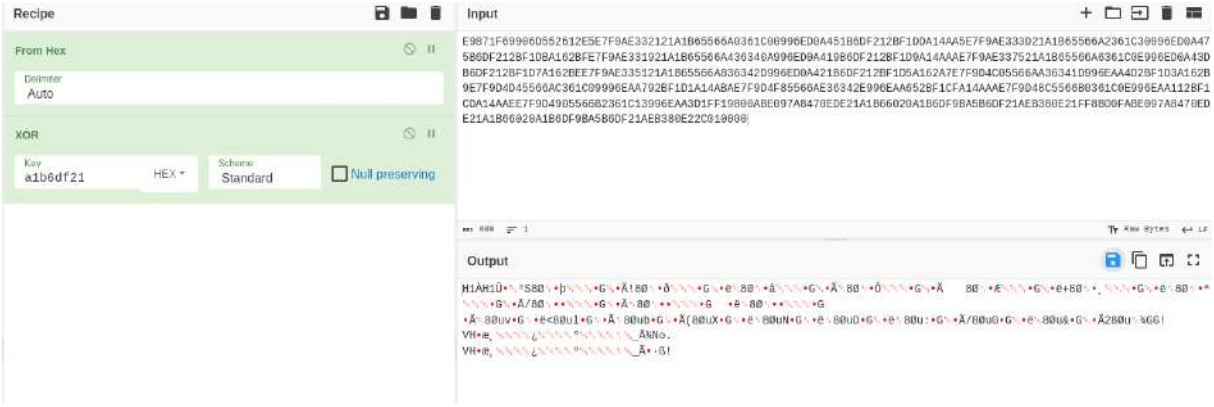
La véritable copie se fait uniquement grâce à l'instruction *rep movsq*. Avant cette instruction, rsi est la source, rdi est la destination (c'est d'ailleurs pour cela qu'il y a un "s" et un "d" dans les noms de ces registres.). rcx est le nombre de répétition de la copie, c'est à dire la taille de la copie. Mais à quoi correspondent toutes ces opérations étranges ?
Puisque v3 est un multiple de 8, en tant qu'adresse renvoyée par mmap (et donc aligné sur une page), ces opérations n'ont aucun effet sur les valeurs.
En réalité, cela provient d'un appel à *__builtin___clear_cache* qui permet d'effacer le cache d'instruction dans le processeur, puisque nous venons de modifier une zone exécutable. Typiquement cela n'est qu'une sécurité afin de s'assurer un comportement déterministe.
Cela ne devrait pas être nécessaire sur une architecture x86, puisque les caches d'instructions sont censé rester cohérent. En revanche cela a été utile au moment de la compilation, puisque qu'il s'agit d'une des méthodes qui permettent d'empêcher GCC de supprimer ce qu'il pense être du code mort.

Toutes ces considérations sortent complètement du cadre du reverse de ce programme. Il faut juste retenir que le code xoré est copié dans une zone exécutable en mémoire, puis lancé. C'est ce qu'on appelle du "packing".
La clé xor fait 4 byte, et dépend uniquement de valeurs hardcodées, ainsi que du retour de la fonction ptrace.

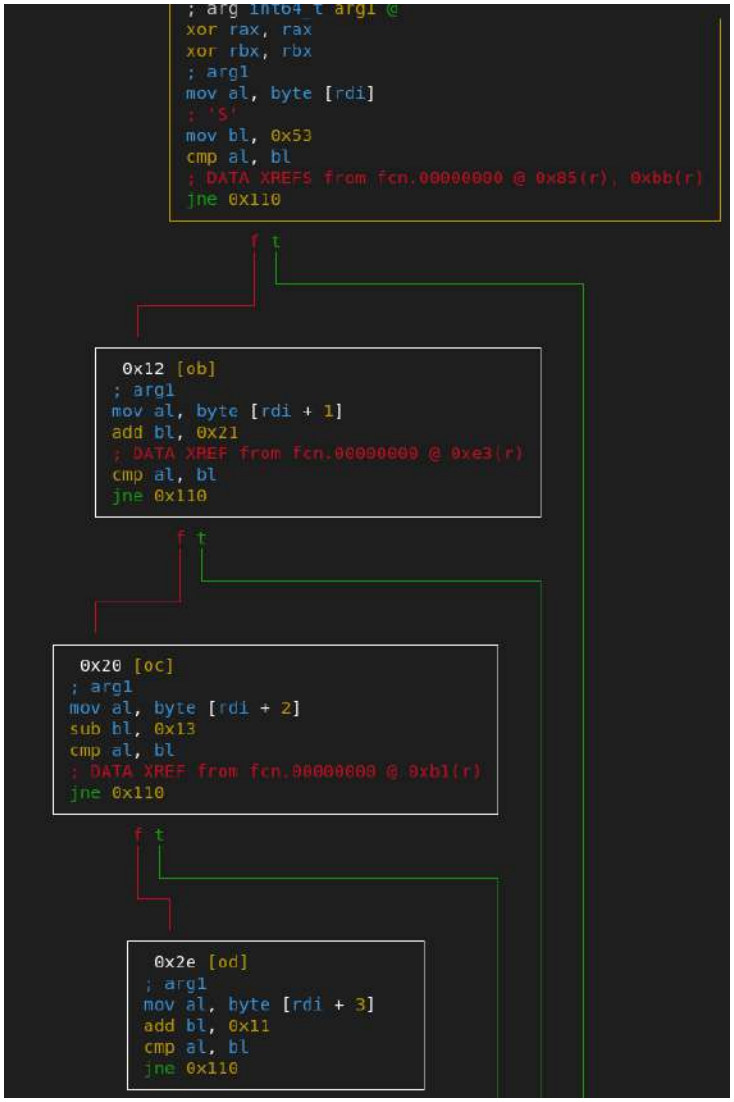
Ptrace est une fonction qui permet de s'attacher à un processus. Les limitations du système d'exploitation font qu'on ne peut s'attacher qu'une fois maximum à un processus. Toute tentative supplémentaire résulte en une erreur (ptrace renvoie -1 plutôt que 0).
Cela a pour effet d'être un anti-debug très basique. En effet les debuggueurs (comme gdb) ont besoin de s'attacher au processus. Ainsi on peut détecter dans son programme si un débuggueur a été attaché (si ptrace renvoie -1 au premier appel).

Avec les 4 appels à ptrace, le comportement normal est donc, 0 pour le premier appel, et -1 pour les autres. La clé xor est finalement "\xa1\xb6\xdf\x21"

Utilisons cyberchef pour déxorer et exporter dans un fichier. (j'ai obtenu les données xorées avec le raccourci shift+E sur IDA, en ayant sélectionné la data à exporter.)



Analysons le shellcode avec radare2. Voici un extrait de la vue panel, qui nous informe sur le fonctionnement du check.



La comparaison se fait caractère par caractère, le premier étant en clair, les suivants se faisant relativement au précédant, à la suite d'une addition ou d'une soustraction. On peut résoudre à la main puisque qu'il n'y a que 20 caractères. Cependant, je vais vous proposer ici une solution automatisée qui utilise miasm.

```python
from miasm.analysis.binary import Container
from miasm.analysis.machine import Machine
from miasm.core.locationdb import LocationDB
from miasm.expression.expression import *

with open('shellcode', 'rb') as f:
    shellcode = f.read()

loc_db = LocationDB()
cont = Container.from_string(
    shellcode,
    loc_db
)

machine = Machine("x86_64")
mdis = machine.dis_engine(cont.bin_stream, loc_db=loc_db)
asmcfg = mdis.dis_multiblock(0)


flag = ''
c = 0
for block in asmcfg.blocks:
    for line in block.lines:
        if line.args and line.args[0] == ExprId('BL', 8):
            if line.name == 'MOV':
                c = int(line.args[1])
                flag += chr(c)
            if line.name == 'ADD':
                c += int(line.args[1])
                flag += chr(c)
            if line.name == 'SUB':
                c -= int(line.args[1])
                flag += chr(c)

print(flag)
```

Le flag est finalement : "Star{P4ck_p4Ck_P4cK}"

4 Challenges de difficulté : Difficile

Si vous êtes là c'est que vous n'êtes plus exactement un débutant, donc je vais moins vous tenir par la main.

4.1 Cave à vin : Muscat (4/4)

Commençons par lire l'énoncé.

Cave à vin : Muscat (4/4)

500

hard

Ce challenge a été tiré du StarHackademINT 2022, puis modifié

Bon, mon ami n'est pas encore au point en reverse. Peut-être que les mathématiques, elles, arriveront à protéger ma cave à vin.

Challenge original : Crackme4 de Redhpm

Auteur du vol : Sckathach

caveavin4

FIGURE 8 – Énoncé du challenge.

C'est un crackme, pas de surprises. Je vais vous montrer 2 solutions, puisqu'il y en a une en one-liner !

Ouvrons le binaire dans IDA. Il est assez simple à comprendre. Globalement, il s'agit du même code en boucle, avec des valeurs qui changent.

Voici un exemple de bloc, j'ai déjà renommé la fonction appelée, nous la verrons ensuite.

```
v3[0] = 0;
v3[1] = 0;
v3[2] = 0;
v3[3] = 1;
v3[4] = 1;
v3[5] = 1;
v3[6] = 0;
v3[7] = 1;
v3[8] = 1;
v3[9] = 1;
v3[10] = 1;
v3[11] = 1;
v3[12] = 0;
v3[13] = 0;
v3[14] = 1;
v3[15] = 0;
v3[16] = 1;
v3[17] = 0;
v3[18] = 1;
v3[19] = 1;
v3[20] = 1;
v3[21] = 1;
v3[22] = 1;
v3[23] = 1;
if ( (unsigned __int8)which_xor(password[1], v3) != 82 )
{
  printf("Too Bad...");
  exit(0);
}
printf("GG!");
exit(0);
```

La fonction *which_xor* permet juste de xorer les caractères du mot de passe entre eux, en fonction d'un tableau qui agit comme sélecteur. S'il y a un 1, alors le caractère à cet indice se retrouve dans le xor.

```
1  __int64 __fastcall which_xor(char *a1, char *a2)
2  {
3    int i; // [rsp+18h] [rbp-8h]
4    unsigned __int8 v4; // [rsp+1Fh] [rbp-1h]
5
6    v4 = 0;
7    for ( i = 0; i <= 23; ++i )
8    {
9      if ( a2[i] )
10       v4 ^= a1[i];
11   }
12   return v4;
13 }
```

Nous sommes donc en présence d'un grand système d'équations, dont nous allons chercher la solution, et certainement pas à la main. Il s'agit du premier exemple dans ce PDF d'un binaire très simple à comprendre, mais non trivial à résoudre.

Je vais d'abord utiliser une bibliothèque python pour résoudre le système, par habitude, je vais utiliser z3.

Tout d'abord j'extrais les données utiles avec r2.

```
aaa
s main
pif~mov byte[3] > bits
pif~cmp al[2] > ress
```

J'ai donc les bits qui correspondent aux choix de xor dans le fichier bits, et les résultats en sortie de *which_xor* dans ress.

Ensuite je n'ai qu'à lire ces fichiers dans python pour reconstruire les équations, et résoudre avec z3.

Voici le script :

```python
import z3

LEN_FLAG = 24

s = z3.Solver()
input_vec = [z3.BitVec(f'input{i}', 8) for i in range(LEN_FLAG)]

def which_xor(s, w):
    res = 0
    for i in range(len(s)):
        if w[i]:
            res ^= s[i]
    return res

system_coefs = []
with open('bits', 'r') as f:
    bits = f.readlines()[::-1]

while bits:
    vec = []
    for i in range(LEN_FLAG):
        vec.append(int(bits.pop()))
    system_coefs.append(vec)

results = []
with open('ress', 'r') as f:
    for res in f.readlines():
        results.append(int(res, 16))

```

```python
for i in range(len(system_coefs)):
    s.add(which_xor(input_vec, system_coefs[i]) == results[i])

print(sat := s.check())
if sat == z3.sat:
    m = s.model()
    print(''.join([chr(m[input_vec[i]].as_long()) for i in range(
   LEN_FLAG)]))
```

Il est également possible de résoudre le challenge avec de l'exécution symbolique, ici avec radius2. Je ne vais pas m'étendre, c'est un teaser pour ma formation exécution symbolique qui arrivera un jour.

```
┌─redhpm@debian in ~/Downloads
└─$ radius2 -p caveavin4 -X Bad -s flag 192 -A . flag -b exit

  flag : "Star{Id34l_p0uR_l_4P3r0}"
```

Le flag est donc : "Star{Id34l_p0uR_l_4P3r0}"

4.2 Disques rayés

Commençons par lire l'énoncé.

FIGURE 9 – Énoncé du challenge.

En téléchargeant le zip donné, on se retrouve avec 100 fichiers binaires et des instructions dans un fichier *notes.txt*.

Les instructions sont :

```
Le contenu du flag est la concatenation de tous les mots de passe
avec un pipe en sha256.

Exemple:
Si le mots de passe des binaires sont a,b, .... z :
Star{sha256(a|b|...|z)}
```

Les binaires sont tous a priori très similaires, au vu de leur taille, identique à l'octet près.

```
└$ ls -l bins
total 1600
-rwxr-xr-x 1 redhpm redhpm 16328 Sep  7 13:49 0.bin
-rwxr-xr-x 1 redhpm redhpm 16328 Sep  7 13:49 10.bin
-rwxr-xr-x 1 redhpm redhpm 16328 Sep  7 13:49 11.bin
-rwxr-xr-x 1 redhpm redhpm 16328 Sep  7 13:49 12.bin
-rwxr-xr-x 1 redhpm redhpm 16328 Sep  7 13:49 13.bin
-rwxr-xr-x 1 redhpm redhpm 16328 Sep  7 13:49 14.bin
-rwxr-xr-x 1 redhpm redhpm 16328 Sep  7 13:49 15.bin
-rwxr-xr-x 1 redhpm redhpm 16328 Sep  7 13:49 16.bin
-rwxr-xr-x 1 redhpm redhpm 16328 Sep  7 13:49 17.bin
-rwxr-xr-x 1 redhpm redhpm 16328 Sep  7 13:49 18.bin
-rwxr-xr-x 1 redhpm redhpm 16328 Sep  7 13:49 19.bin
-rwxr-xr-x 1 redhpm redhpm 16328 Sep  7 13:49 1.bin
```

Il devient alors apparent que nous allons devoir reverse un des binaires, puis développer un programme qui automatise la récupération du mot de passe pour chacun des binaires.

Ouvrons alors un des binaires avec IDA, pour voir ce dont il s'agit.

La première partie est un simple input de 30 caractères maximum, avec un nullbyte ajouté à la fin.

```
16   fgets(s, 30, stdin);
17   for ( i = 0; ; ++i )
18   {
19     if ( i > 29 )
20       goto LABEL_6;
21     if ( s[i] == 10 )
22       break;
23   }
24   s[i] = 0;
25 LABEL_6:
```

Il n'y a rien à ajouter sur ce point. Ensuite la taille du mot de passe est vérifiée avec un simple *strlen*.

```
26   if ( strlen(s) != apericube )
27     exit(0);
```

Passons au coeur du programme. Voici ici une version intégralement nettoyée. J'ai renommé les variables de sorte à correspondre à leur utilité. J'ai appliqué la structure *user_regs_struct* sur la variable qui sauvegarde les registres. J'ai redimensionné les tableaux de sorte à faire sens.

```
29  pid = fork();
30  if ( pid )                                          // father
31  {
32    received_len = 0;
33    while ( 1 )
34    {
35      waitpid(pid, &stat_loc, 0);
36      if ( (stat_loc & 0x7F) == 0 )
37        break;
38      ptrace(PTRACE_GETREGS, (unsigned int)pid, 0LL, &regs);
39      ciphered_received[received_len++] = regs.rax;
40      ptrace(PTRACE_CONT, (unsigned int)pid, 0LL, 0LL);
41    }
42    ciphered_received[received_len] = 0;
43    is_valid = 1;
44    for ( j = 0; j < (int)received_len; ++j )
45      is_valid = ((unsigned __int8)(ciphered_received[j] ^ ciphered_password[j]) == 0) & (unsigned __int8)is_valid;
46    if ( is_valid )
47    {
48      gg_shellcode = (void (__fastcall *)(void *))decipher(
49                                                  &ciphered_gg_shellcode,
50                                                  (unsigned int)ciphered_gg_shellcode_len,
51                                                  password,
52                                                  received_len);
53      gg_shellcode(&ciphered_gg_shellcode);
54    }
55  }
56  else                                                // child
57  {
58    ptrace(PTRACE_TRACEME, 0LL, 0LL, 0LL);
59    send_to_father = (void (__fastcall *)(char *))decipher(
60                                                  chipered_send_to_father,
61                                                  (unsigned int)ciphered_send_to_father_len,
62                                                  key,
63                                                  (unsigned int)key_len);
64    send_to_father(password);
65  }
66  exit(0);
67 }
```

Passons à une explication du programme. Le *fork* sépare l'exécution en 2 processus. Le père et le fils ne vont pas faire la même tâche. Ici le père s'attache au fils, pour récupérer les registres du fils. Cette opération est effectuée quand une interruption est reçue du fils (avec le *waitpid*).

Ce principe est classique, c'est ce qu'on appelle des "nanomites". Il s'agit d'une méthode d'antidebug très efficace quand implémentée correctement. En effet, la communication par ptrace est **nécessaire** au bon fonctionnement du programme. Il devient alors impossible d'attacher un débuggueur en plus, sans empêcher cette communication.

Commençons donc par analyser le fils, statiquement, pour savoir quelles sont les informations envoyées au père.

Ici, un shellcode est unpacké puis lancé avec le mot de passe en paramètre. Voyons comment fonctionne ce packing, c'est à dire la fonction de déchiffrement.

La voici dans IDA, nettoyée.

```
1 char *__fastcall decipher(const char *data, int data_len, char *key, int key_len)
2 {
3   unsigned __int8 v7; // [rsp+29h] [rbp-17h]
4   unsigned __int8 v8; // [rsp+29h] [rbp-17h]
5   unsigned __int8 v9; // [rsp+2Ah] [rbp-16h]
6   char v10; // [rsp+2Bh] [rbp-15h]
7   char v11; // [rsp+2Bh] [rbp-15h]
8   int i; // [rsp+2Ch] [rbp-14h]
9   int j; // [rsp+2Ch] [rbp-14h]
10  int k; // [rsp+2Ch] [rbp-14h]
11  _BYTE *pseudo_random_stream; // [rsp+30h] [rbp-10h]
12  char *shellcode; // [rsp+38h] [rbp-8h]
13
14  pseudo_random_stream = malloc(0x100uLL);
15  shellcode = (char *)mmap(0LL, 0x1000uLL, 7, 34, 0, 0LL);
16  memcpy(shellcode, data, data_len);
17  for ( i = 0; i <= 255; ++i )
18    pseudo_random_stream[i] = i;
19  v7 = 0;
20  for ( j = 0; j <= 255; ++j )
21  {
22    v7 += pseudo_random_stream[j] + key[j % key_len];
23    v10 = pseudo_random_stream[v7];
24    pseudo_random_stream[v7] = pseudo_random_stream[(unsigned __int8)j];
25    pseudo_random_stream[(unsigned __int8)j] = v10;
26  }
27  v8 = 0;
28  v9 = 0;
29  for ( k = 0; k < data_len; ++k )
30  {
31    v9 += pseudo_random_stream[++v8];
32    v11 = pseudo_random_stream[v8];
33    pseudo_random_stream[v8] = pseudo_random_stream[v9];
34    pseudo_random_stream[v9] = v11;
35    shellcode[k] ^= pseudo_random_stream[(unsigned __int8)(pseudo_random_stream[v8] + pseudo_random_stream[v9])];
36  }
37  return shellcode;
38 }
```

La fonction commence par générer un flux d'octets pseudo aléatoires en fonction de la clé. Ensuite la donnée est xorée par ce flux. Ici 2 possibilités :

— On reconnaît l'algo.
— On le reverse à la main.

Quand j'ai résolu le challenge, je n'ai pas reconnu l'algorithme. Mais ce n'est pas très important, puisqu'il est trivial de le réimplémenter. (Pour votre information, il s'agit d'un RC4).

Déchiffrons donc à la main le shellcode du binaire courant, afin de voir comment il marche.

J'extrais donc les données et je les déchiffres avec python. Ensuite je mets les données brutes dans un fichier que j'ouvre avec r2.

```
23: fcn.00000000 (int64_t arg1);
        ; arg int64_t arg1 @ rdi
        0x00000000      4831c0          xor rax, rax
        0x00000003      4835cd000000    xor rax, 0xcd                ; 205
        0x00000009      488b1f          mov rbx, qword [rdi]         ; arg1
        0x0000000c      4831d8          xor rax, rbx
        0x0000000f      4883c04a        add rax, 0x4a
        0x00000013      48ffc7          inc rdi                      ; arg1
        0x00000016      cc              int3
        0x00000017      488b1f          mov rbx, qword [rdi]
        0x0000001a      4831d8          xor rax, rbx
        0x0000001d      4883c02e        add rax, 0x2e
        0x00000021      48ffc7          inc rdi
        0x00000024      cc              int3
        0x00000025      488b1f          mov rbx, qword [rdi]
        0x00000028      4831d8          xor rax, rbx
        0x0000002b      4883c03e        add rax, 0x3e
        0x0000002f      48ffc7          inc rdi
        0x00000032      cc              int3
        0x00000033      488b1f          mov rbx, qword [rdi]
        0x00000036      4831d8          xor rax, rbx
        0x00000039      4805ac000000    add rax, 0xac                ; 172
        0x0000003f      48ffc7          inc rdi
        0x00000042      cc              int3
        0x00000043      488b1f          mov rbx, qword [rdi]
        0x00000046      4831d8          xor rax, rbx
        0x00000049      4883c02a        add rax, 0x2a
        0x0000004d      48ffc7          inc rdi
        0x00000050      cc              int3
        0x00000051      488b1f          mov rbx, qword [rdi]
        0x00000054      4831d8          xor rax, rbx
        0x00000057      4883c00d        add rax, 0xd
        0x0000005b      48ffc7          inc rdi
        0x0000005e      cc              int3
        0x0000005f      488b1f          mov rbx, qword [rdi]
        0x00000062      4831d8          xor rax, rbx
        0x00000065      4805d5000000    add rax, 0xd5                ; 213
        0x0000006b      48ffc7          inc rdi
        0x0000006e      cc              int3
        0x0000006f      488b1f          mov rbx, qword [rdi]
        0x00000072      4831d8          xor rax, rbx
        0x00000075      4805cb000000    add rax, 0xcb                ; 203
        0x0000007b      48ffc7          inc rdi
        0x0000007e      cc              int3
        0x0000007f      488b1f          mov rbx, qword [rdi]
        0x00000082      4831d8          xor rax, rbx
        0x00000085      4805e3000000    add rax, 0xe3                ; 227
        0x0000008b      48ffc7          inc rdi
        0x0000008e      cc              int3
        0x0000008f      4831c0          xor rax, rax
        0x00000092      4831ff          xor rdi, rdi
        0x00000095      83f03c          xor eax, 0x3c
        0x00000098      0f05            syscall
```

Puisque *rdi* contient le premier argument, c'est à dire l'adresse du mot de passe, nous assistons à une procédure de chiffrement caractère par caractère. Entre chaque caractères une interruption de type "Breakpoint" est lancée au père, avec l'instruction *int*3.

Ok on a bien compris le fonctionnement du fils. Passons au père.

Quand une interruption est reçue, le père sauvegarde le registre *rax* dans un tableau. Ensuite une boucle vérifie que les données reçues sont égales à de la donnée hardcodée. Cette boucle est légèrement obfusquée, puisqu'elle vérifie que tous les xors font 0. Cela reste simple.

Si cette vérification est remplie, un nouveau shellcode est déchiffré, avec cette fois le mot de passe comme clé. A ce stade, on peut supposer qu'il affiche un simple message de félicitation.

On a donc dans le binaire le mot de passe chiffré par le shellcode du fils ! Il suffit de récupérer le shellcode pour chaque binaire, afin d'avoir les données nécessaires au déchiffrement du mot de passe chiffré !

Le programme doit donc :

— Récupérer les données nécessaires au déchiffrement du shellcode (donnée et clé, avec leur longueurs).
— Récupérer le mot de passe chiffré.
— Parser le shellcode pour déchiffrer le mot de passe.

J'ai fait le choix de hardcoder les adresses des données puisque tous les binaires sont les mêmes. Le parsisng du shellcode est fait à l'aide de miasm, comme vu dans un autre write-up.

Voici mon script final :

```
import binascii

from miasm.analysis.binary import Container
from miasm.analysis.machine import Machine
from miasm.core.locationdb import LocationDB
from miasm.expression.expression import *

import logging
logging.disable(logging.CRITICAL)

def decipher(buf, key):
    buf = list(buf)
    rkey = list(range(0x100))

```

```
    j = 0
    for i in range(0x100):
        j += rkey[i] + key[i % len(key)]
        j &= 0xff
        rkey[j], rkey[i] = rkey[i], rkey[j]

    j = 0
    cpt = 0
    for k in range(len(buf)):
        cpt += 1
        cpt &= 0xff
        j += rkey[cpt]
        j &= 0xff
        rkey[j], rkey[cpt] = rkey[cpt], rkey[j]
        buf[k] ^= rkey[(rkey[cpt] + rkey[j]) & 0xff]

    return bytes(buf)

def extract_vals(bs):
    vals = []
    loc_db = LocationDB()
    cont = Container.from_string(
        bs,
        loc_db
    )

    machine = Machine("x86_64")
    mdis = machine.dis_engine(cont.bin_stream, loc_db=loc_db)

    asmcfg = mdis.dis_multiblock(0)
    val = 0
    for block in asmcfg.blocks:
        for line in block.lines:
            if line.name == 'XOR':
                val = line.args[1]
            elif line.name == 'ADD':
                val = line.args[1]
            elif line.name == 'SYSCALL':
                break
            elif isinstance(val, ExprInt):
                vals.append(val.arg)
                val = 0

    return vals

def compute_input(ciph, vals):
    inp = b''
    for i in range(len(ciph)):
        c = (((ciph[i] - vals[1]) % 256) ^ vals[0]) & 0xff
```

```python
        inp += c.to_bytes(1, 'big')
        vals = [ciph[i]] + vals[2:]

    return inp

def get_pass(payload, key, goal):

    deciphered = decipher(binascii.unhexlify(payload), key)

    return compute_input(binascii.unhexlify(goal), extract_vals(deciphered))

def get_data(path):
    with open(path, 'rb') as f:
        d = f.read()

    k_add = d[0x3090] + d[0x3091] * 256
    p_add = d[0x3088] + d[0x3089] * 256
    payload = d[p_add:p_add + d[0x3074] + d[0x3075] * 256]
    key = d[k_add:k_add + d[0x3078] + d[0x3079] * 256]
    goal = d[0x2008:0x2008 + d[0x3070]]

    return binascii.hexlify(payload).decode(), key, binascii.hexlify(goal).decode()

passes = []
for i in range(100):
    passes.append(get_pass(*get_data(f'./{i}.bin')).decode())

with open('output.txt', 'w') as f:
    print('|'.join(passes), end='', file=f)
```

Ensuite on peut calculer le flag comme indiqué dans les instructions.

```
┌─redhpm@debian in ~/Downloads/dr/bins
└─$ sha256sum output.txt
dbc1d8204ab92504f810feafdd655fcf7e4474734df7813ae59d9d2ebcb4578e  output.txt
```

Le flag est donc :
”Star{dbc1d8204ab92504f810feafdd655fcf7e4474734df7813ae59d9d2ebcb4578e}”